QA QUEST

BIBLIOTECA DE ESTUDOS / MATERIAL DE APOIO

Testes de API

Aprenda a interpretar requisições, validar respostas e comparar o comportamento de APIs com contratos.

01 Requisitar

02 Comparar

03 Validar

TRILHA 04 · PRÁTICA PROFISSIONAL

MÓDULO 02 · TESTES DE API

Guia introdutório · 6 páginas · v1.0 · 01/11/2026

Material complementar. Não contém respostas das missões.

01 / FUNDAMENTOS

O que significa testar uma API?

Uma interface pode parecer correta mesmo quando o serviço responde errado.

Uma **API** permite que sistemas troquem informações e acionem operações por interfaces definidas. Em uma API HTTP, um cliente envia uma **requisição** e recebe uma **resposta**. O teste compara ambas com o contrato: método, rota, parâmetros, dados, status, cabeçalhos e regras de negócio.

Da requisição à resposta



O que cada parte representa?

Elemento	Exemplo e finalidade
Método e rota	GET /api/horarios consulta horários; POST /api/reservas solicita criação.
Parâmetros	?data=2026-11-12 limita a consulta a uma data.
Body da requisição	JSON com os campos necessários à operação.
Status e headers	Indicam resultado e formato, como Content-Type: application/json.
Body da resposta	Traz os dados que os consumidores precisam interpretar.

REGRA PRINCIPAL

200 OK não prova que a resposta está correta. A API pode retornar sucesso e ainda omitir um campo, quebrar um filtro ou violar uma regra do negócio.

02 / ESCOPO E CONTRATOS

Planeje antes de executar

Defina o comportamento esperado para evitar interpretar respostas no improviso.

Um **contrato de API** descreve como consumir uma operação e o que esperar. Pode estar em documentação, OpenAPI ou critérios de aceitação. Antes de testar, identifique quais campos são obrigatórios, seus tipos, os status esperados e as regras envolvidas.

Classes de status que você verá

Classe	Interpretação	Exemplos
2xx	Operação bem-sucedida.	200 OK; 201 Created.
4xx	A requisição não pôde ser atendida nas condições enviadas.	400 Bad Request; 404 Not Found.
5xx	Falha atribuída ao processamento no servidor ou intermediário.	500; 503.

O **status exato depende do contrato e da semântica HTTP**. Nem toda operação POST exige 201, nem todo dado inválido exige o mesmo código entre APIs.

Matriz mínima de cenários

Cenário	Verificações sugeridas
Sucesso	Status, formato, campos e valores esperados.
Filtros	Somente registros compatíveis com os parâmetros.
Dados inválidos	Resposta e mensagem de erro previstas no contrato.
Recurso ausente	Tratamento coerente para identificador inexistente.
Acesso	Autorização adequada com contas fictícias de teste.

BOAS PRÁTICAS

Prepare dados isolados, preserve o estado entre execuções e evite credenciais ou dados pessoais reais. Em testes automatizados, valide também os efeitos persistidos quando o cenário exigir.

03 / EXEMPLO INDEPENDENTE

Caso guiado: API AgendaClara

Exemplo didático de uma plataforma fictícia de agendamento de salas.

Contrato de exemplo: **GET /api/horarios?data=2026-11-12** deve retornar **200** com uma lista **horarios**. Cada item possui id (texto), data (texto no padrão YYYY-MM-DD) e disponivel (booleano). Todos os itens precisam corresponder à data solicitada.

Requisição e resposta hipotéticas

```
GET /api/horarios?data=2026-11-12
HTTP/1.1 200 OK
Content-Type: application/json

{ "horarios": [
  { "id": "H-01", "data": "2026-11-12",
    "disponivel": true }
] }
```

O que devemos conferir?

Verificação	Resultado esperado
HTTP	Status 200 e formato JSON conforme contrato.
Estrutura	horarios é uma lista; cada item possui os campos exigidos.
Tipo	id e data são textos; disponivel é booleano.
Negócio	Todos os itens pertencem à data informada no filtro.

Se a API devolve 200, mas apresenta um horário de outro dia, a consulta **não atende à regra**. Esse exemplo é independente da FictFvr e não revela as respostas da missão.

EVIDÊNCIA ÚTIL

Registre método, rota, parâmetros, ambiente, versão, status, campos relevantes e o contraste entre resultado esperado e observado. Remova dados sensíveis dos registros.

04 / AUTOMAÇÃO INTRODUTÓRIA

Automatize verificações pequenas

Comece com assertions objetivas; a ferramenta não substitui o raciocínio de teste.

Postman permite criar testes em JavaScript após uma resposta. No exemplo abaixo, o script é colocado na aba **Scripts → Post-response** de uma requisição configurada para o endpoint fictício. O código só funciona quando existe uma resposta real ou simulada da ferramenta.

Exemplo de assertions com Postman

```
pm.test("HTTP 200", () => {
  pm.response.to.have.status(200);
});

const body = pm.response.json();
pm.test("Lista de horarios valida", () => {
  pm.expect(body.horarios).to.be.an("array");
  body.horarios.forEach((h) => {
    pm.expect(h.data).to.eql("2026-11-12");
    pm.expect(h.disponivel).to.be.a("boolean");
  });
});
```

Esse teste verifica apenas os pontos expressos nas afirmações. Ele **não comprova automaticamente** autenticação, regras de reserva, persistência, desempenho ou todos os casos negativos.

Escolha a ferramenta para a tarefa

Ferramenta	Aplicação introdutória
Postman	Explorar requisições e escrever assertions em coleções.
Playwright	Testar endpoints e preparar/verificar estado em testes automatizados.
Cypress	Executar requisições HTTP e validar respostas com cy.request().

ATENÇÃO

Execute somente contra APIs próprias, fictícias ou autorizadas. Não use tokens reais nos exemplos, no repositório ou em screenshots.

05 / PRÁTICA E REFERÊNCIAS

Sua vez: construa uma análise

Atividade rápida para praticar antes de abrir o módulo do QA Quest.

Na API fictícia **BibliotecaViva**, o contrato diz que **GET /api/livros?genero=ficcao** deve retornar 200, uma lista de livros e somente registros cujo campo genero seja ficcao. Imagine que uma resposta retorna 200 com um livro de outro gênero.

Desafio de revisão

- Identifique método, rota, parâmetro, status e body a conferir.
- Escreva **três assertions** que validariam o contrato.
- Separe resultado esperado, observado e impacto no consumidor.
- Proponha **um cenário negativo** e um dado de teste seguro.

Checklist antes de concluir

- [ ] Validei o status sem ignorar o conteúdo da resposta.
- [ ] Comparei campos, tipos e regras com o contrato.
- [ ] Cobri pelo menos um caso negativo relevante.
- [ ] Registrei evidências reprodutíveis e sem segredos.

Referências gratuitas para aprofundar

Documentação pública e gratuita. Clique nos títulos para consultar as fontes:

01 **MDN · Códigos de status HTTP**

02 **Postman Docs · Escrever testes de resposta**

03 **OpenAPI Initiative · Especificação oficial**

04 **Playwright Docs · API testing**

05 **Cypress Docs · API testing**